

Operating Systems Project

Kristi Gorea PhD Cand.

Coursework details

Title	Simulation of Contiguous Memory Allocation Techniques
Type	Group work (3-4 students per group)
Contribution	20% of the course final grade
Hand-out date	07/01/2026
Submission date	06/02/2026

Description / Scenario

One method commonly used by researchers and OS designers to evaluate memory allocation policies is through simulations of memory management algorithms. In contiguous memory allocation, processes are allocated blocks of consecutive memory, and different strategies can significantly affect fragmentation and utilization.

In this coursework, students are required to provide a simple implementation, in a high level programming language of their choice, of three contiguous memory management algorithms: First-Fit, Best-Fit, and Worst-Fit, assuming single-partition processes and a single contiguous main memory region. Students may use either a Command Line or Graphical Interface.

Task I

Implement a program that simulates all three contiguous allocation algorithms above (or, alternatively, one separate program for each algorithm), which interactively obtains the simulation data from the user, consisting of:

- Total memory size (e.g., 1000 units).
- A sequence of operations:
 - Allocation requests: process ID and requested memory size (integer units).
 - Deallocation requests: process ID to be freed.

Assumptions:

- Single contiguous main memory region starting empty (one free hole).
- Each process occupies exactly one contiguous block.

For every run of the simulation, the program must output at least:

- The memory layout after each operation, showing allocated segments (PID, start, size) and free holes.
- Whether each allocation request succeeds or fails for the chosen algorithm.
- Summary statistics at the end of the run:

- Number of successful allocations and failures.
- Total free memory and size of the largest free hole (as a basic fragmentation indicator).

The program must support running the same sequence of operations under all three algorithms (First-Fit, Best-Fit, Worst-Fit) for comparison.

Example run and output snippet (Task I)

Example scenario (FIRST-FIT):

- Total memory: 1000 units
- Operations:
 1. A P1 130
 2. A P2 60
 3. A P3 200
 4. F P2
 5. A P4 50
 6. A P5 300
 7. F P1
 8. A P6 120

Algorithm: FIRST-FIT

Total memory: 1000

Initial state:

Free holes:

[0, 999] (size = 1000)

1) A P1 130

-> Allocation SUCCESS

Memory layout:

P1 [0, 129] size = 130

HOLE [130, 999] size = 870

2) A P2 60

-> Allocation SUCCESS

Memory layout:

P1 [0, 129] size = 130

P2 [130, 189] size = 60

HOLE [190, 999] size = 810

3) A P3 200

-> Allocation SUCCESS

Memory layout:

P1 [0, 129] size = 130

P2 [130, 189] size = 60

P3 [190, 389] size = 200

HOLE [390, 999] size = 610

4) F P2

-> Deallocation SUCCESS

Memory layout:

P1 [0, 129] size = 130

HOLE [130, 189] size = 60

P3 [190, 389] size = 200

HOLE [390, 999] size = 610

5) A P4 50

-> Allocation SUCCESS (placed into hole [130, 189])

Memory layout:

P1 [0, 129] size = 130

P4 [130, 179] size = 50

HOLE [180, 189] size = 10

P3 [190, 389] size = 200

HOLE [390, 999] size = 610

6) A P5 300

-> Allocation SUCCESS

Memory layout:

P1 [0, 129] size = 130

P4 [130, 179] size = 50

HOLE [180, 189] size = 10

P3 [190, 389] size = 200

P5 [390, 689] size = 300

HOLE [690, 999] size = 310

7) F P1

-> Deallocation SUCCESS

Memory layout:

HOLE [0, 129] size = 130

P4 [130, 179] size = 50

HOLE [180, 189] size = 10

P3 [190, 389] size = 200

P5 [390, 689] size = 300

HOLE [690, 999] size = 310

8) A P6 120

-> Allocation SUCCESS (placed into hole [0, 129])

Memory layout:

P6 [0, 119] size = 120

HOLE [120, 129] size = 10

P4 [130, 179] size = 50

HOLE [180, 189] size = 10

P3 [190, 389] size = 200

P5 [390, 689] size = 300

HOLE [690, 999] size = 310

Final statistics (FIRST-FIT):

Successful allocations: 5

Failed allocations: 0

Total free memory: 340

Number of holes: 3

Largest hole size: 310

Memory utilization: $660 / 1000 = 66.0\%$

Task II

Based on your implementation from Task I, extend the program so that it can automatically run simple experiments with randomly generated processes. This version is intentionally light and focuses on a few ranges and very simple statistics.

The program should:

- Ask the user for:
 - Total memory size (e.g., 1000 units).
 - Number of processes n to generate (e.g., 10 or 20).
- For each of the following two ranges of process sizes:
 - Range 1: 10 to 50 units
 - Range 2: 50 to 150 units

For each range and each algorithm (First-Fit, Best-Fit, Worst-Fit), do the following once:

- Generate n processes with random sizes in the given range.
- Try to allocate all of them in order (no deallocations are required in this light version).
- At the end, print for that algorithm and range:
 - Number of successful allocations.
 - Number of failed allocations.
 - Total allocated memory.
 - Total free memory.

Example :

=== Range 10–50 ===

Number of processes: 10

Total memory: 1000

Algorithm: FIRST-FIT

Successful allocations: 8

Failed allocations: 2

Total allocated: 310

Total free: 690

Algorithm: BEST-FIT

Successful allocations: 9

Failed allocations: 1

Total allocated: 340

Total free: 660

Algorithm: WORST-FIT

Successful allocations: 7

Failed allocations: 3

Total allocated: 280

Total free: 720

In the report, students briefly answer in 1–2 short paragraphs:

- For these simple tests, which algorithm tends to fit more processes for each range?
- Can they relate this observation to the way each algorithm chooses a free hole?

Submission

Students are expected to submit:

- **Short report** documenting the results. It should describe the high level logic and design of the program(s), how it is installed / executed and how it is used, with sample screenshots. For point b) it should describe the results collected from the simulation. The report must be submitted electronically in the instructor's email address (kgorea@unyt.edu.al) before the deadline.
- **Program code**, compressed in a zip file and sent by email to instructor's address (kgorea@unyt.edu.al) before the deadline.

Evaluation Criteria

- Report structure, completeness and clarity.
- Program completeness and accuracy in implementation of the three algorithms. Proper code documentation through comments is also taken into consideration.
- Task I contributes 70% to the coursework grade, while Task II contributes 30%.

Important Notice

- **This coursework is subject to the STUDENT HONOUR CODE. The University's rules on academic dishonesty (e.g. cheating, plagiarism, submitting false information) will be strictly enforced.**
- **It is on the discretion of the instructor to ask the student to give a demonstration of the program or to come for a question session about the work she/he submitted.**
- **Late submissions are accepted with the following penalties: -10% for each day late.**